

Assignment on Pandas

Exploratory Data Analysis (EDA)

Q1. Creating and Inspecting DataFrames

Create a CSV file named `students.csv` with the following data. Then load it using pandas and inspect the DataFrame.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [20, 22, 21, 23, 20],
    'Marks': [85, 78, 92, 88, 75],
    'Subject': ['Math', 'Science', 'Math', 'Science', 'Math']
}
df = pd.DataFrame(data)
df.to_csv('students.csv', index=False)

df = pd.read_csv('students.csv')
```

Perform the following tasks:

- Display the entire DataFrame to see all rows and columns.
- Use `shape` to find the total number of rows and columns.
- Use `columns` to print all column names.
- Use `dtypes` to check the data type of each column.
- Use `info()` to see column names, non-null counts, and memory usage.
- Use `describe()` to get statistical summary for numeric columns.
- Use `head(3)` to display only the first 3 rows.
- Use `tail(2)` to display only the last 2 rows.

Q2. Data Selection and Indexing

Create a CSV file named `matrix_data.csv` with the following data. Then load it and practice selecting specific rows, columns, and values using different pandas methods.

```
import pandas as pd

df = pd.DataFrame({
    'A': [10, 20, 30, 40, 50],
    'B': [15, 25, 35, 45, 55],
    'C': ['x', 'y', 'z', 'x', 'y']
}, index=['r1', 'r2', 'r3', 'r4', 'r5'])
df.to_csv('matrix_data.csv')

df = pd.read_csv('matrix_data.csv', index_col=0)
```

Perform the following tasks:

- Select column A using bracket notation: `df['A']`.
- Select columns A and B together: `df[['A', 'B']]`.
- Select row with label 'r3' using `loc['r3']`.
- Select row at position 2 using `iloc[2]`.
- Select the single value at row 'r4', column 'B' using `loc['r4', 'B']`.
- Select rows 'r2' to 'r4' and columns 'A' to 'B' using `loc['r2':'r4', 'A':'B']`.
- Select rows at positions 1 to 3 and columns at positions 0 to 1 using `iloc[1:4, 0:2]`.
- Select all rows where column A values are greater than 25 using boolean filtering.
- Select rows where column C is either 'x' or 'z' using `isin(['x', 'z'])`.
- Select rows where column B is between 20 and 45 (inclusive) using `between(20, 45)`.

Q3. Handling Missing Data

Create a CSV file named `missing_data.csv` with missing values. Then load it and practice detecting, counting, and handling missing data.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'Age': [20, np.nan, 21, 23, np.nan, 22],
    'Marks': [85, 78, np.nan, 88, 75, np.nan],
    'Subject': ['Math', 'Science', 'Math', np.nan, 'Math', 'Science']
})
df.to_csv('missing_data.csv', index=False)

df = pd.read_csv('missing_data.csv')
```

Perform the following tasks:

- Use `isnull()` to create a boolean mask showing True where values are missing.
- Use `isnull().sum()` to count missing values in each column.
- Use `isnull().sum().sum()` to get the total number of missing values in the entire DataFrame.
- Use `dropna()` to remove all rows containing any missing value. Store in a new variable and count how many rows remain.
- Use `dropna(how='all')` to remove only rows where all values are missing.
- Fill missing Age values with the mean of the Age column using `fillna()`.
- Fill missing Marks values with the median of the Marks column.
- Fill missing Subject values with the mode (most frequent value) of the Subject column.
- Explain in 2-3 sentences: When should you use `dropna()` versus `fillna()` during EDA?

Q4. Data Cleaning: Duplicates, Types, and Renaming

Create a CSV file named `cleaning_data.csv` with duplicate rows and incorrect data types. Then clean it by removing duplicates, converting types, and renaming columns.

```
import pandas as pd

df = pd.DataFrame({
    'Student_ID': ['S001', 'S002', 'S003', 'S001', 'S004', 'S002'],
    'Join_Date': ['2023-01-15', '2023-02-20', '2023-03-10',
                  '2023-01-15', '2023-04-05', '2023-02-20'],
    'Score': ['85.5', '78.0', '92.3', '85.5', '88.1', '78.0'],
    'Passed': ['Yes', 'No', 'Yes', 'Yes', 'Yes', 'No']
})
df.to_csv('cleaning_data.csv', index=False)

df = pd.read_csv('cleaning_data.csv')
```

Perform the following tasks:

- Use duplicated() to identify which rows are duplicates.
- Use duplicated().sum() to count the total number of duplicate rows.
- Use drop_duplicates() to remove duplicate rows. Store the result in a new DataFrame and display it.
- Convert the 'Score' column from string to float using astype(float).
- Convert the 'Join_Date' column to datetime format using pd.to_datetime().
- Convert the 'Passed' column to boolean type.
- Rename the column 'Student_ID' to 'ID' and 'Join_Date' to 'Enrollment_Date' using rename().
- Use replace() to change all 'Yes' values to 1 and all 'No' values to 0 in the Passed column.
- Explain in 2-3 sentences: Why is data type conversion important before performing statistical analysis?

Q5. Sorting and Ranking

Create a CSV file named sorting_data.csv with student marks and attendance. Then practice sorting rows and ranking students based on their performance.

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'Marks': [85, 78, 92, 88, 75, 91],
    'Attendance': [90, 85, 95, 80, 88, 92]
})
df.to_csv('sorting_data.csv', index=False)

df = pd.read_csv('sorting_data.csv')
```

Perform the following tasks:

- Sort the DataFrame by the 'Marks' column in ascending order using sort_values().
- Sort the DataFrame by the 'Marks' column in descending order.
- Sort by multiple columns: first by 'Attendance' (ascending), then by 'Marks' (descending).
- Add a new column 'Rank' that ranks students by Marks where rank 1 = highest marks. Use rank(method='min', ascending=False).
- Add a column 'Attendance_Rank' using the dense ranking method.

- Find the top 3 students by Marks using `nlargest(3, 'Marks')`.
- Find the bottom 2 students by Attendance using `nsmallest(2, 'Attendance')`.
- After sorting, reset the index to default integers and drop the old index using `reset_index(drop=True)`.

Q6. GroupBy Operations

Create a CSV file named `employee_data.csv` with employee records across different departments. Then use `groupby` to summarize data by categories.

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['Sales', 'IT', 'Sales', 'HR', 'IT', 'HR', 'Sales', 'IT'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace', 'Henry'],
    'Salary': [50000, 70000, 55000, 45000, 75000, 48000, 60000, 72000],
    'Experience': [2, 5, 3, 4, 6, 3, 4, 7]
})
df.to_csv('employee_data.csv', index=False)

df = pd.read_csv('employee_data.csv')
```

Perform the following tasks:

- Group by 'Department' and calculate the mean Salary for each department.
- Group by 'Department' and find the total Salary expenditure for each department.
- Group by 'Department' and count the number of employees in each department using `size()`.
- Group by 'Department' and calculate min, max, and mean of Experience using `agg()` with a list.
- Group by 'Department' and apply multiple aggregations on different columns: mean Salary and sum Experience. Use a dictionary inside `agg()`.
- Use `transform()` to add a new column 'Dept_Avg_Salary' that shows the average salary of each employee's department.
- Filter and display only those departments where the average salary is greater than 55000.
- Explain in 2-3 sentences: What is the difference between aggregation (`agg`) and transformation (`transform`) in `GroupBy`?

Q7. Statistical Analysis and Value Counts

Create a CSV file named `product_data.csv` with sales and rating information. Then calculate various statistics and analyze value distributions.

```
import pandas as pd

df = pd.DataFrame({
    'Product': ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C', 'A'],
    'Sales': [120, 200, 150, 180, 220, 130, 190, 210, 160, 175],
    'Rating': [4.2, 3.8, 4.5, 4.0, 3.9, 4.3, 4.1, 4.0, 4.4, 3.7],
    'Region': ['East', 'West', 'East', 'North', 'West', 'South',
               'East', 'West', 'South', 'North']
})
```

```
df.to_csv('product_data.csv', index=False)

df = pd.read_csv('product_data.csv')
```

Perform the following tasks:

- Calculate mean, median, and standard deviation of the Sales column.
- Calculate mean, median, and standard deviation of the Rating column.
- Find the minimum and maximum values in the Sales column.
- Calculate the 25th, 50th, and 75th percentile of Sales using quantile().
- Compute the correlation matrix for all numeric columns using corr(). Identify which two columns are most correlated.
- Compute the covariance matrix for numeric columns using cov().
- Use value_counts() on the Product column to count how many times each product appears.
- Use value_counts(normalize=True) on the Region column to get the percentage distribution of each region.
- Count the number of unique values in each column using nunique().
- Calculate the range of Sales (max minus min).

Q8. Data Transformation: Apply, Map, and Replace

Create a CSV file named grades_data.csv with student marks and letter grades. Then transform the data by applying custom functions and mapping values.

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Marks': [85, 78, 92, 88, 75],
    'Grade_Letter': ['B', 'C', 'A', 'A', 'C']
})
df.to_csv('grades_data.csv', index=False)

df = pd.read_csv('grades_data.csv')
```

Perform the following tasks:

- Use apply() on the Marks column to create a new column 'Marks_Squared' containing the square of each mark.
- Use apply() with a lambda function to create a column 'Category' that categorizes marks: 'High' if ≥ 85 , 'Medium' if 70-84, 'Low' if < 70 .
- Use map() on the Grade_Letter column to convert letters to grade points: A=4, B=3, C=2, D=1, F=0. Store in a new column 'Grade_Points'.
- Use replace() to replace all occurrences of the name 'Charlie' with 'Chris' in the Name column.
- Use apply() on the entire DataFrame to convert all string values to uppercase.
- Explain in 2-3 sentences: What is the difference between apply(), map(), and applymap()? When should you use each one?

Q9. Merging, Joining, and Concatenating

Create three CSV files with related data, then practice combining them using `merge()` and `concat()`.

```
import pandas as pd

df1 = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana']
})
df1.to_csv('employees.csv', index=False)

df2 = pd.DataFrame({
    'ID': [3, 4, 5, 6],
    'Salary': [50000, 60000, 70000, 80000]
})
df2.to_csv('salaries.csv', index=False)

df3 = pd.DataFrame({
    'ID': [7, 8],
    'Name': ['Eve', 'Frank'],
    'Salary': [55000, 65000]
})
df3.to_csv('new_employees.csv', index=False)

df1 = pd.read_csv('employees.csv')
df2 = pd.read_csv('salaries.csv')
df3 = pd.read_csv('new_employees.csv')
```

Perform the following tasks:

- Merge `df1` and `df2` on the 'ID' column using an inner join (`how='inner'`). How many rows result? Explain why.
- Merge `df1` and `df2` on 'ID' using a left join (`how='left'`). Which rows are included? Which have NaN values?
- Merge `df1` and `df2` on 'ID' using a right join (`how='right'`). Compare the result with the left join.
- Merge `df1` and `df2` on 'ID' using an outer join (`how='outer'`). How many rows result? Explain why outer join includes more rows.
- Concatenate `df1` and `df3` vertically (row-wise) using `pd.concat([df1, df3], axis=0)`.
- Concatenate `df1` and `df2` horizontally (column-wise) with `axis=1`. What do you observe about the alignment of rows?
- Explain in 2-3 sentences: What is the difference between `merge()` and `concat()`? When would you use each?
- Explain in 2-3 sentences: When should you use an inner join versus an outer join during EDA?

Q10. Pivot Tables and Cross Tabulation

Create a CSV file named `sales_data.csv` with sales and profit data across regions and products. Then create pivot tables and cross-tabulations to summarize the data.

```
import pandas as pd

df = pd.DataFrame({
```

```
'Region': ['North', 'South', 'East', 'West', 'North',  
'South', 'East', 'West', 'North', 'South'],  
'Product': ['A', 'A', 'B', 'B', 'A', 'B', 'A', 'B', 'B', 'A'],  
'Sales': [200, 150, 300, 250, 180, 220, 280, 260, 190, 210],  
'Profit': [50, 30, 80, 60, 45, 55, 70, 65, 40, 52]  
})  
df.to_csv('sales_data.csv', index=False)  
  
df = pd.read_csv('sales_data.csv')
```

Perform the following tasks:

- Create a pivot table showing the average Sales by Region (rows) and Product (columns).
- Create a pivot table showing the total Profit by Region (rows) and Product (columns).
- Create a pivot table with multiple aggregations: mean Sales and sum Profit, grouped by Region.
- Use `pd.crosstab()` to create a frequency table showing the count of each Product for each Region.
- Use `pd.crosstab()` with `normalize=True` to show proportions instead of raw counts.
- Add `margins=True` to the `crosstab` to display row totals and column totals.
- Explain in 2-3 sentences: When is `pivot_table()` more useful than `groupby()` in EDA?
- Explain in 2-3 sentences: What is the difference between a pivot table and a cross-tabulation (`crosstab`)?